

# IF3230 – Sistem Paralel dan Terdistribusi Fault Tolerant dan Consensus

Achmad Imam Kistijantoro ([imam@staff.stei.itb.ac.id](mailto:imam@staff.stei.itb.ac.id))

Robithoh Annur ([robithoh@staff.stei.itb.ac.id](mailto:robithoh@staff.stei.itb.ac.id))

Andreas Bara Timur ([bara@staff.stei.itb.ac.id](mailto:bara@staff.stei.itb.ac.id))

# Fault Tolerant

---

- ▶ **Partial Failure**

- ▶ Failure yang terjadi pada salah satu komponen sistem terdistribusi
- ▶ Tidak terjadi pada single-machine system

- ▶ An **important** goal in distributed systems design is to construct the system in such a way that it can **automatically recover** from **partial failures** without seriously affecting the overall performance.



# Fault Tolerance Terms

---

**Fail** : system cannot meet its promises

**Error** : a part of a system's state that may lead to a failure

**Fault** : the cause of an error

## Fault tolerance

- ▶ System can provide its services even in the presence of faults
- ▶ Dependable systems

## Type of fault :

- ▶ Transient : hanya terjadi satu kali kemudian hilang
- ▶ Intermittent : terjadi berulang kali
- ▶ Permanent : terjadi terus menerus hingga komponen diganti



# Dependable Systems

---

## 4 requirement of Dependable Systems (Kopetz and Verissimo, 1993)

- ▶ **Availability**
  - ▶ System is ready to be used immediately
- ▶ **Reliability**
  - ▶ system can run continuously without failure
- ▶ **Safety**
  - ▶ situation that when a system temporarily fails to operate correctly, nothing catastrophic happens
- ▶ **Maintainability**
  - ▶ how easy a failed system can be repaired



# Failure Models

---

| Type of failure                                                             | Description                                                                                                                      |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Crash failure                                                               | A server halts, but is working correctly until it halts                                                                          |
| Omission failure<br><i>Receive omission</i><br><i>Send omission</i>         | A server fails to respond to incoming requests<br>A server fails to receive incoming messages<br>A server fails to send messages |
| Timing failure                                                              | A server's response lies outside the specified time interval                                                                     |
| Response failure<br><i>Value failure</i><br><i>State transition failure</i> | A server's response is incorrect<br>The value of the response is wrong<br>The server deviates from the correct flow of control   |
| Arbitrary failure                                                           | A server may produce arbitrary responses at arbitrary times                                                                      |



# Failure Detection

---

- ▶ To properly mask failures, we need to detect them
- ▶ 2 mechanisms:
  - ▶ Actively send “are you alive” message to each other
  - ▶ Passively wait message come in from other process
- ▶ There has been a huge body of **theoretical work** on failure detectors
  - ▶ **Timeout** mechanism is used to check whether a process has failed.



# Failure Masking by Redundancy

---

Jika failure terjadi, sedapat mungkin failure disembunyikan dari proses lain/user

- ▶ Key technique for masking faults is to use redundancy
- ▶ Information redundancy
  - ▶ Adding extra bits to allow recovery from garbled bits
  - ▶ Hamming code
- ▶ Time redundancy
  - ▶ Mengulangi action
  - ▶ Berguna untuk fault jenis transient atau intermitten
- ▶ Physical redundancy
  - ▶ Adding extra processes or extra component to replace fault component



# Failure Masking and Replication

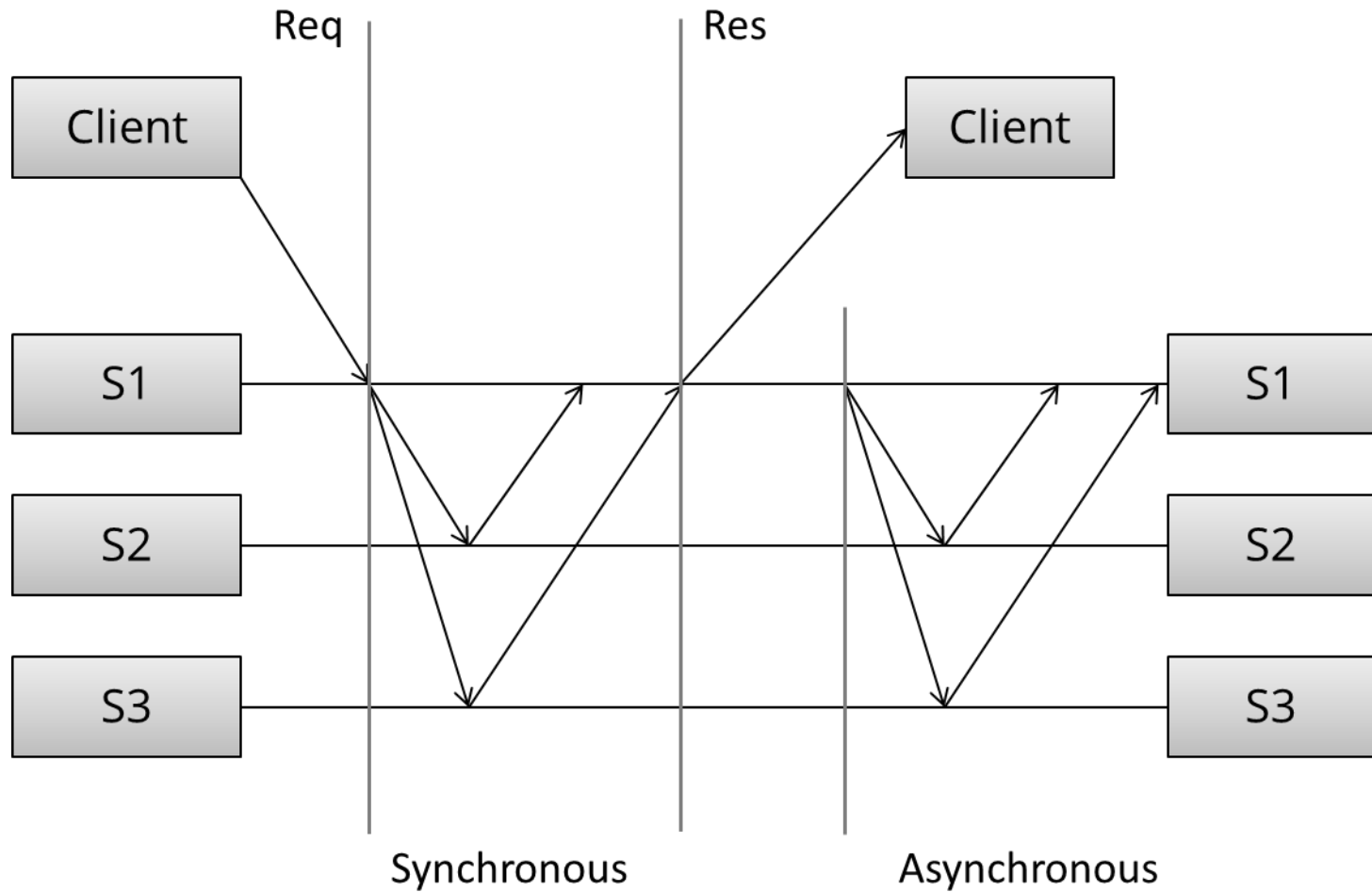
---

Pendekatan utama untuk menangani proses yang faulty adalah dengan menjalankan beberapa proses yang identik dalam sebuah grup

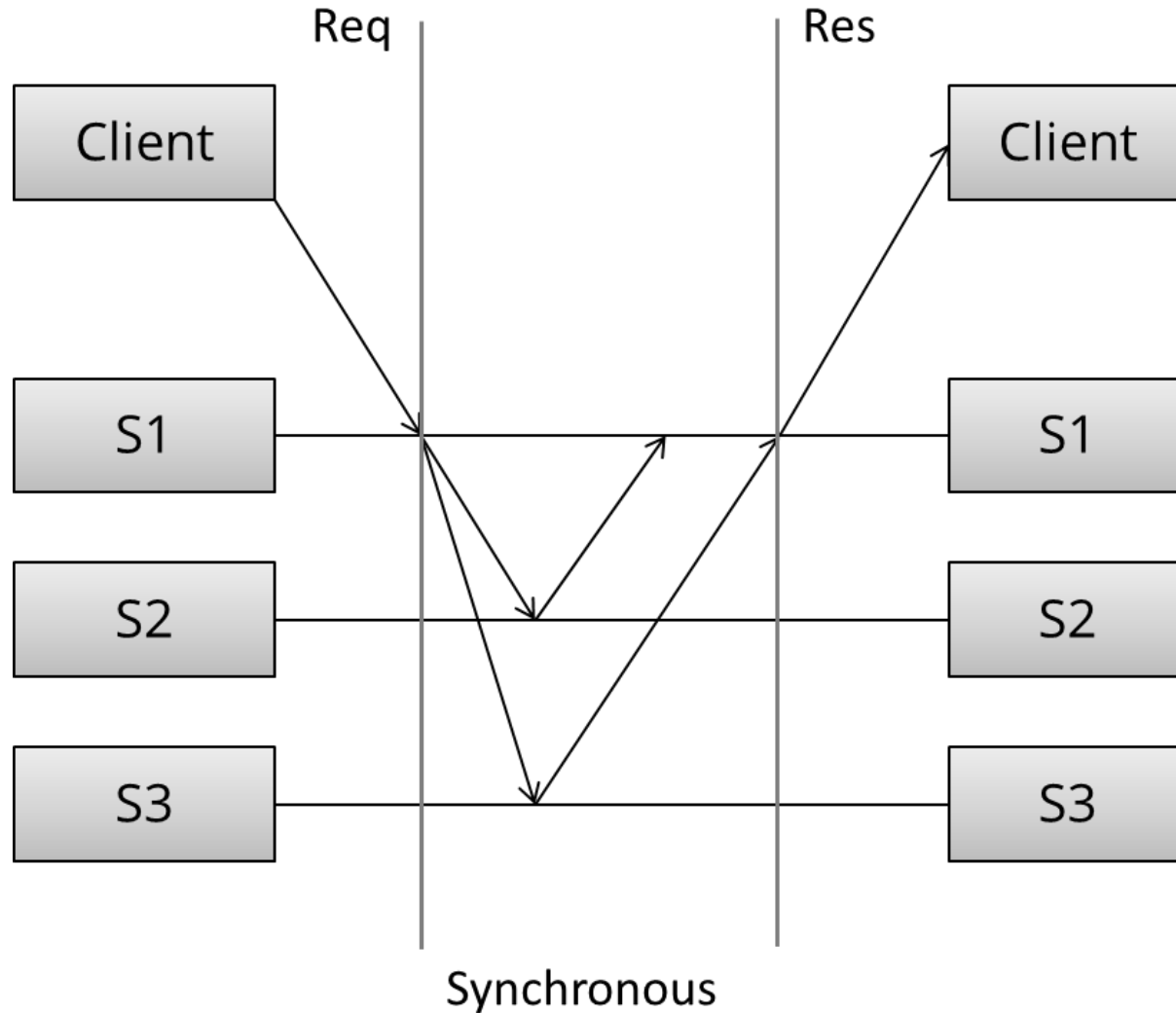
- ▶ saat sebuah pesan dikirim ke grup, semua anggota grup menerimanya
- ▶ Jika salah satu proses pada grup fail, ada proses lain yang dapat mengambil alih
- ▶ Sebuah sistem disebut  **$k$  fault tolerant** jika dia dapat survive faults pada  $k$  components dan tetap berjalan sesuai spesifikasi
  - ▶ Untuk proses secara umum, dengan  $k + 1$  backup process cukup untuk menyediakan  $k$  fault tolerance.
  - ▶ Untuk proses tertentu (misal: paxos), minimum  $2k + 1$  correct process dibutuhkan untuk mencapai  $k$  fault tolerance.



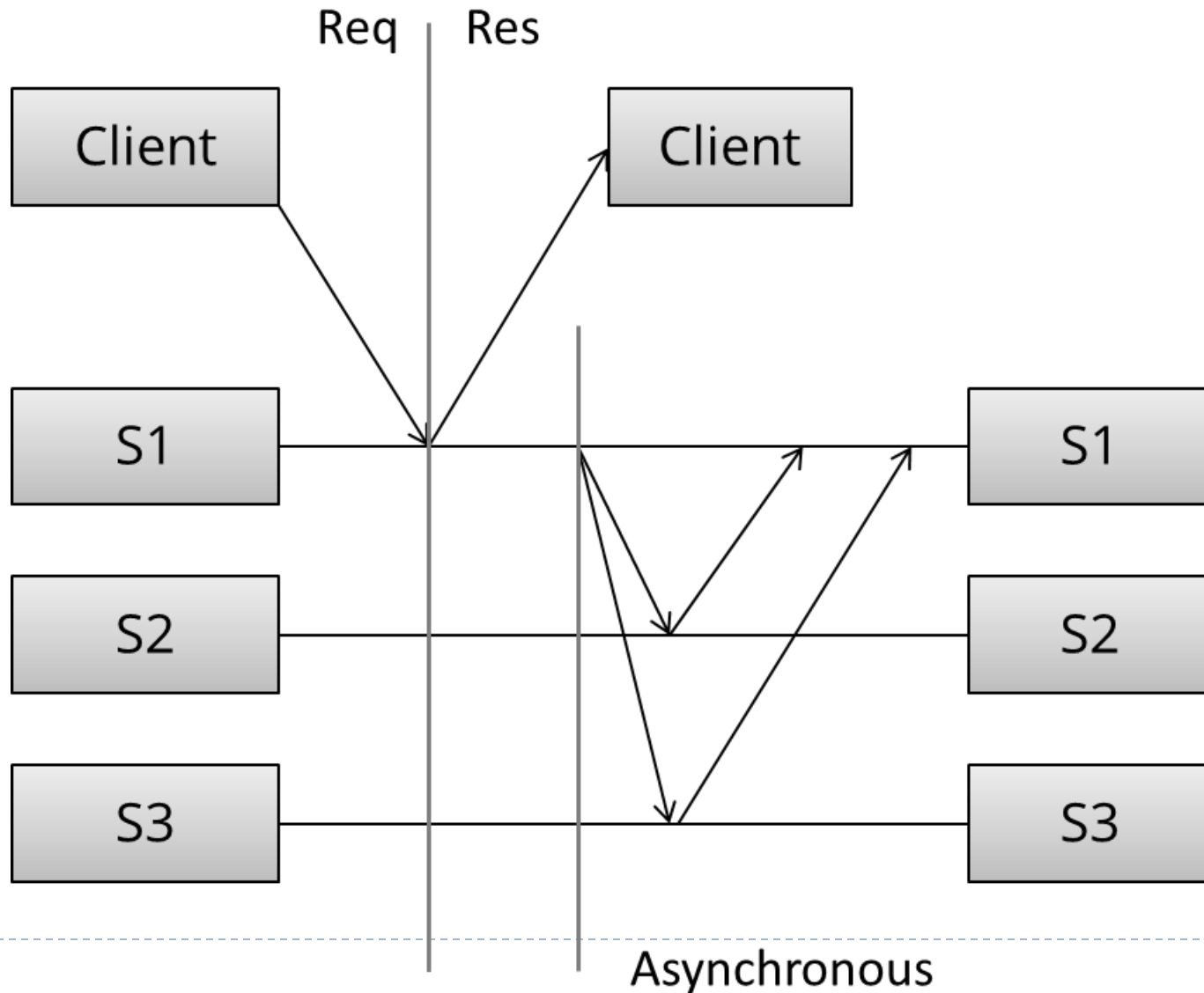
# Replikasi



# Synchronous Replication (active, eager, push, pessimistic)



# Asynchronous replication (lazy, passive, pull, optimistic)



# Replikasi Primary/Backup

---

- ▶ Sering disebut Master/Slave replication, Log Shipping replication
- ▶ Update dilakukan pada master/primary, dan log operasi (atau state change) dikirimkan ke replika backup
- ▶ Ada 2 variant:
  - ▶ asynchronous primary backup replication
  - ▶ synchronous primary backup replication
- ▶ Asynchronous primary/backup berpotensi kehilangan update, jika master fail sebelum mengirimkan backup
- ▶ Synchronous primary/backup berpotensi bermasalah jika master fail sesaat sebelum memberikan ack ke client, namun backup sudah commit
- ▶ **Split-brain** terjadi jika antara primary dan backup terputus koneksi



# Consensus

---

- ▶ Failure masking dapat dilakukan dengan redundancy
  - ▶ Misal menambah jumlah server untuk meningkatkan performansi dan memastikan sistem tetap berjalan meskipun terjadi failure pada suatu server
  - ▶ Perlu sinkronisasi agar semua server memiliki pemahaman yang sama (agreement) atas suatu nilai
- ▶ Solusi : consensus
  - ▶ Contoh algoritma consensus: Paxos



# Consensus Goal

---

Allow a group of processes to agree on a result

- ▶ All processes must agree on the same value
- ▶ The value must be one that was submitted by at least one process (the consensus algorithm cannot just make up a value)
- ▶ **Consensus requirements**
  - ▶ **Validity:** Only proposed values may be selected
  - ▶ **Uniform agreement:** No two nodes may select different values
  - ▶ **Integrity:** A node can select only a single value
  - ▶ **Termination** (progress): Every node will eventually decide on a value



## 2 Phase Commit

---

- ▶ umum digunakan pada database
- ▶ memastikan partisipan sepakat atau tidak untuk commit
- ▶ Phase 1:
  - ▶ koordinator: prepare
  - ▶ partisipan: Ok/Not Ok
- ▶ Phase 2:
  - ▶ koordinator: decision (abort/commit)
  - ▶ partisipan: Ack dan menerapkan decision
- ▶ berpotensi blocking jika koordinator fail saat phase 2



# Paxos

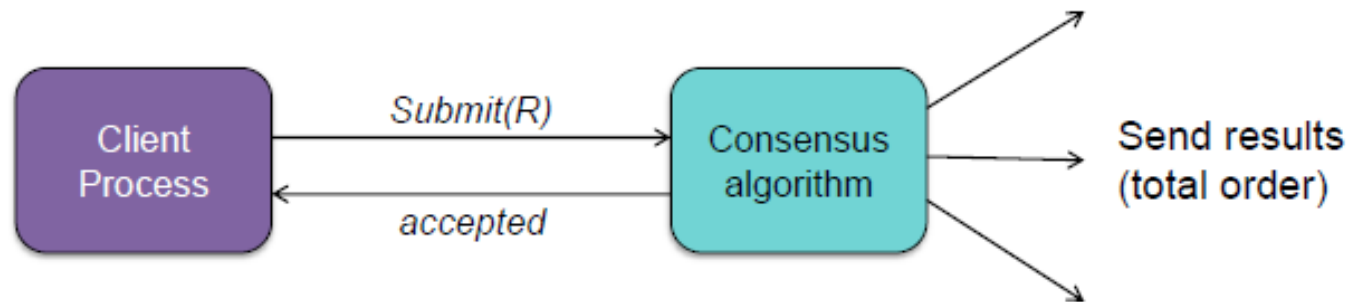
---

- ▶ **Fault-tolerant distributed consensus algorithm**
  - ▶ tidak block jika mayoritas proses berjalan
  - ▶ algoritma membutuhkan mayoritas  $(2P+1)$  proses berjalan untuk menangani kegagalan  $P$  proses
- ▶ **Goal: menyediakan konsisten event order untuk multiple client**
  - ▶ Setiap mesin menjalankan algoritma agree pada proposed value dari client
  - ▶ semua value akan diasosiasikan dengan event/action
  - ▶ paxos menjamin tidak ada mesin yang mengasosiasikan value dengan event lain
- ▶ **Abortable consensus**
  - ▶ permintaan client dapat ditolak
  - ▶ client harus mengirimkan ulang request



# Programmer's View

---



```
while (submit_request(R) != ACCEPTED) ;
```

Think of R as a key:value pair in a database

# Paxos Player

---

- **Client**: makes a request
- **Proposers**:
  - Get a request from a client and run the protocol
  - **Leader**: elected coordinator among the proposers (not necessary but simplifies message numbering and ensures no contention) – we don't count on the presence of a single leader
- **Acceptors**:
  - Multiple processes that remember the state of the protocol
  - **Quorum** = any majority of acceptors
- **Learners**:
  - When agreement has been reached by acceptors, a Learner executes the request and/or sends a response back to the client

*These different roles are usually part of the same system*



# What Paxos Does

---

- **Paxos ensures a consistent ordering in a cluster of machines**
  - Events are ordered by sequential event IDs ( $N$ )
- Client wants to log an event: sends request to a Proposer
  - E.g., *value*,  $v$  = “add \$100 to my checking account”
- **Proposer**
  - Increments the latest event ID it knows about
    - ID = sequence number
  - Asks all the acceptors to reserve that event ID
- **Acceptors**
  - A majority of acceptors have to accept the requested event ID



# Proposal Numbers

---

- Each proposal has a number (created by proposer)
  - Must be unique (e.g., `<sequence#>.<process_id>`)
- Newer proposals take precedence over older ones
- Each acceptor
  - Keeps track of the largest number it has seen so far
- Lower proposal numbers get rejected
  - Acceptor sends back the {number, value} of the currently accepted proposal
  - Proposer has to “play fair”:
    - It will ask the acceptors to accept the {number, value}
    - Either its own or the one it got from the acceptor



# Prinsip Paxos

---

- ▶ **Problem:**

- ▶ decisions in all views must agree.

- ▶ **Key property ensured by Paxos:**

- ▶ once a view reaches a decision, every *view that* supersedes it will either:
    - ▶ reach no decision;
    - ▶ reach the same decision.



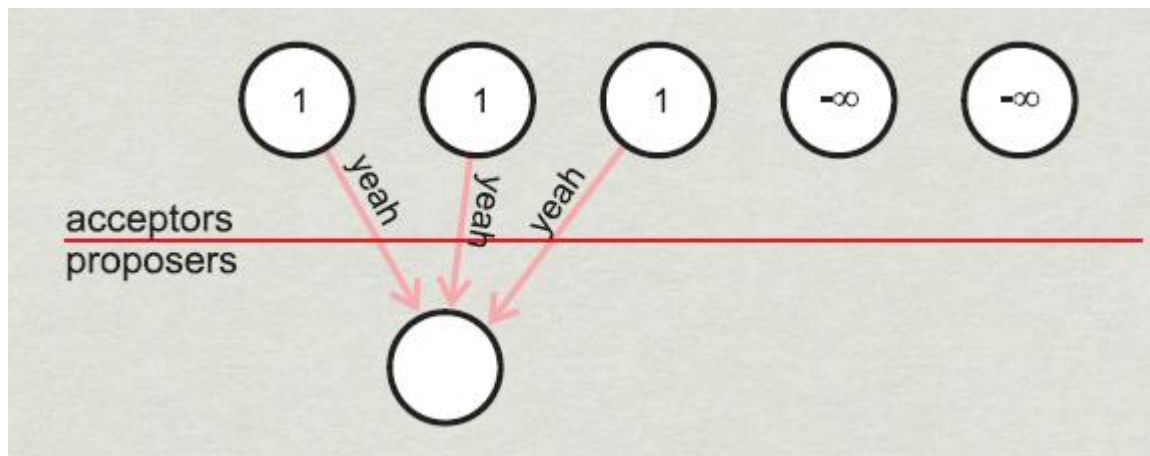
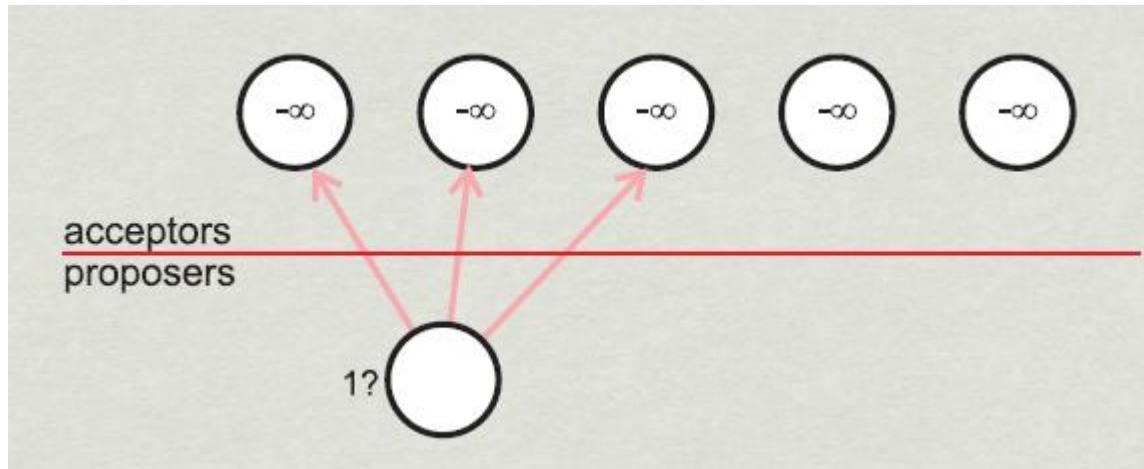
# Paxos – 1st phase

---

- ▶ Paxos is a 2-phase protocol (but it's asynchronous).
- ▶ A proposer, in the first phase:
  - ▶ proposes a **view number to a majority of acceptors**;
  - ▶ view numbers are unique across the whole system;
  - ▶ view numbers are monotonically increasing.
- ▶ An acceptor, in the first phase;
  - ▶ **accepts: if it's the largest number it's ever seen**;
  - ▶ **rejects it otherwise.**
  - ▶ On acceptance, replies with:
    - ▶ the largest view number in which it has accepted a value;
    - ▶ the last value it has accepted (will make sense in a second);
  - ▶ On rejection, might send the largest view value it committed to.

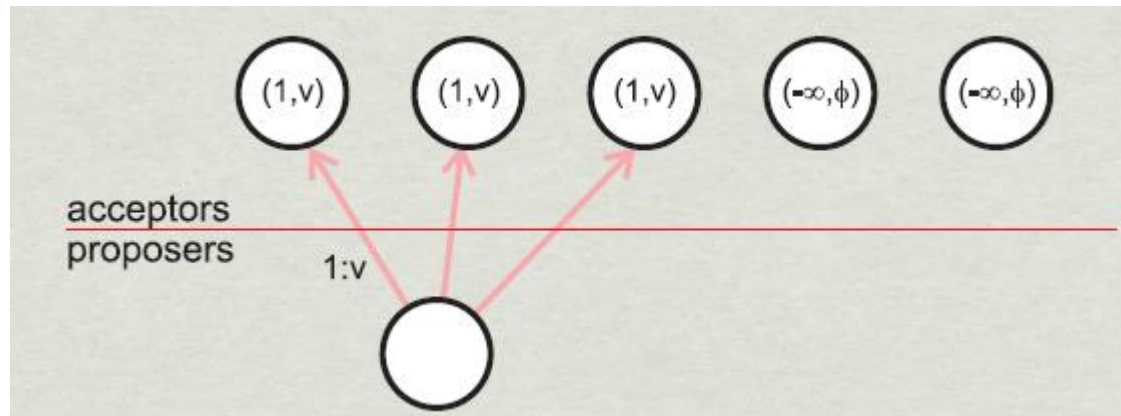


# Paxos 1st phase



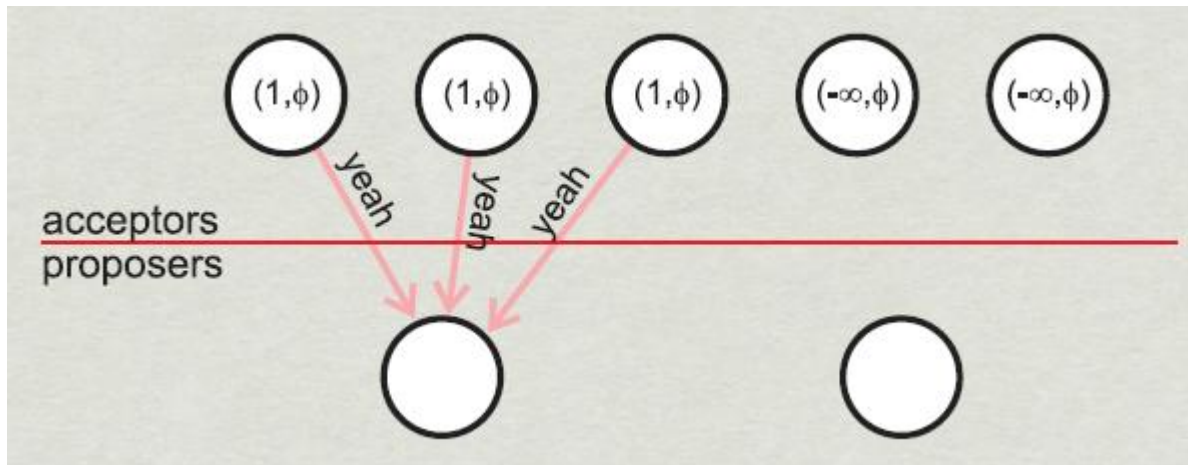
# Paxos 2nd phase

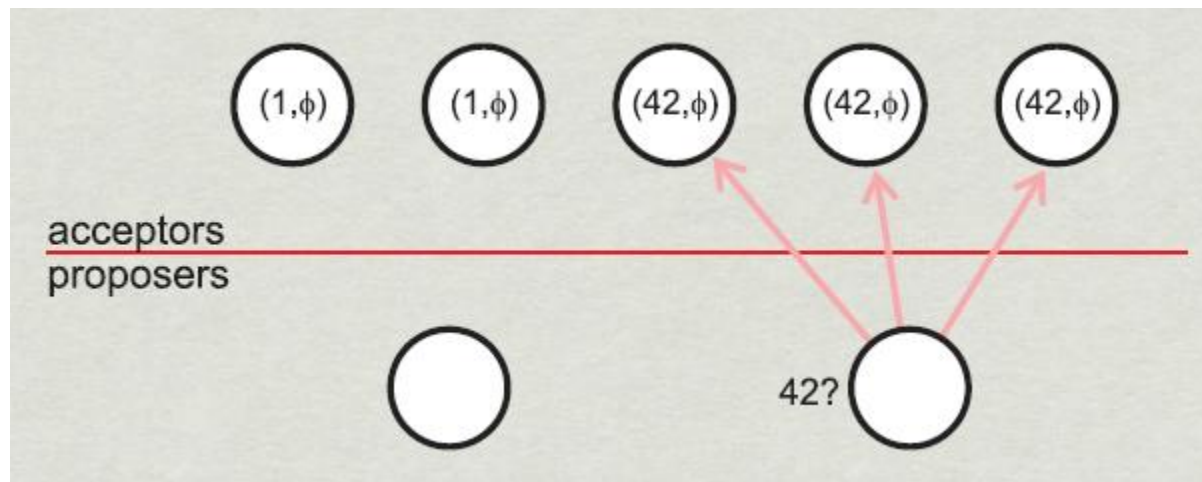
- ▶ If a majority of acceptors answers “yes”;
- ▶ proposer proposes a value to a majority of acceptors;
- ▶ if a majority accepts it, the value is **decided**

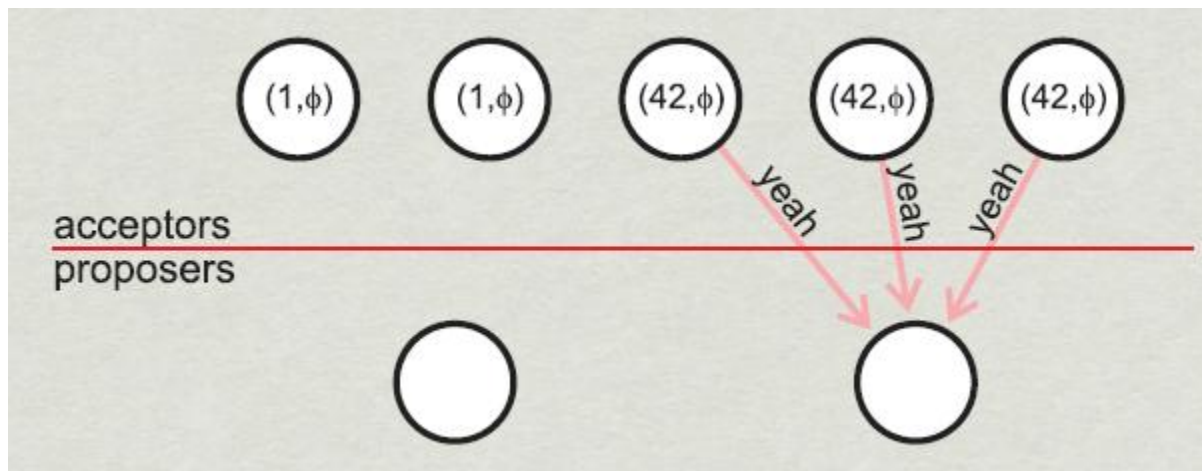


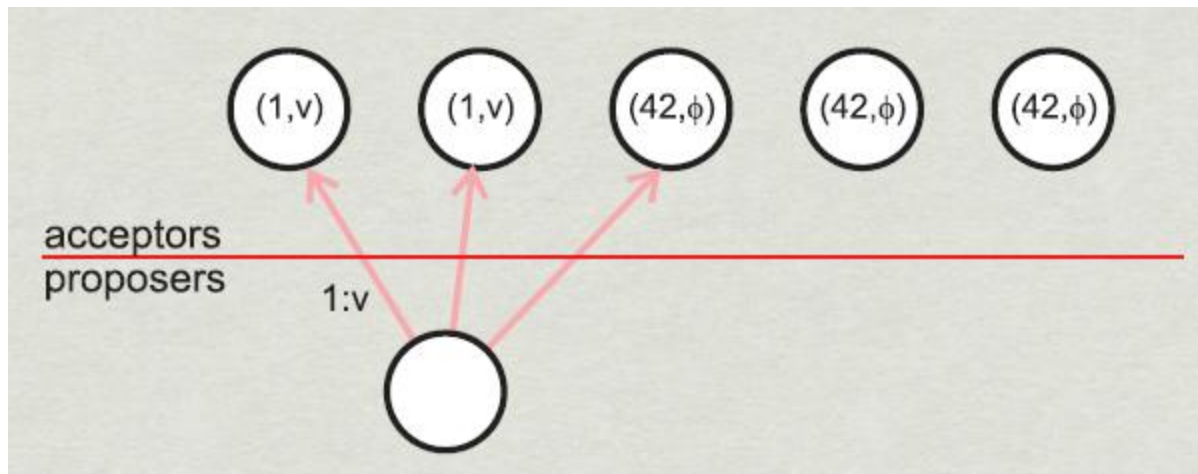
# Paxos 2nd phase

- ▶ Acceptors only accept a value if no **larger view** number has been proposed in the meantime.







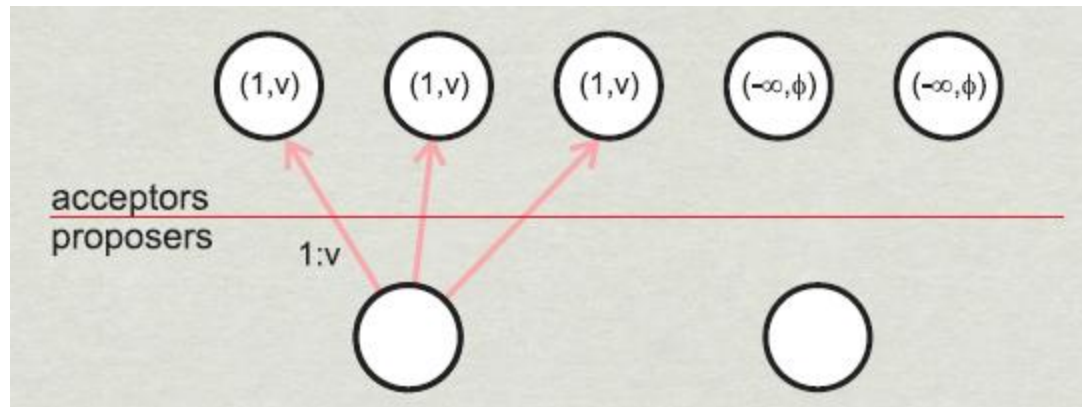


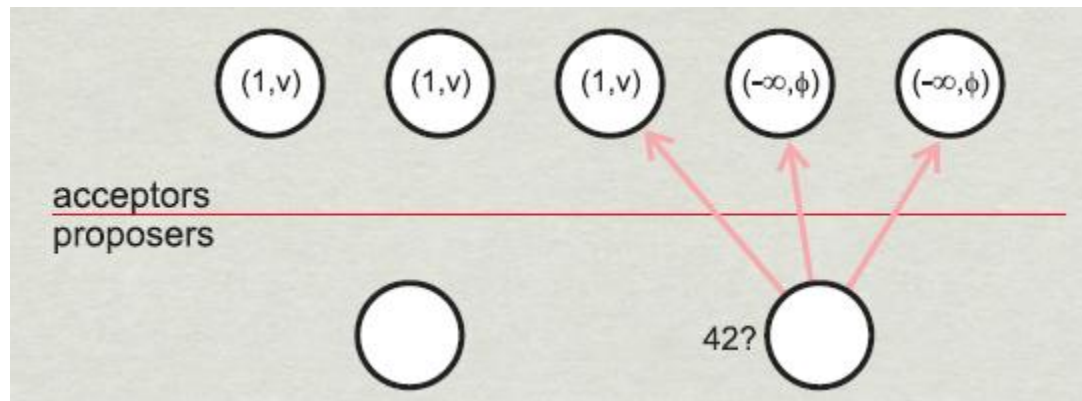
# Paxos 2nd phase

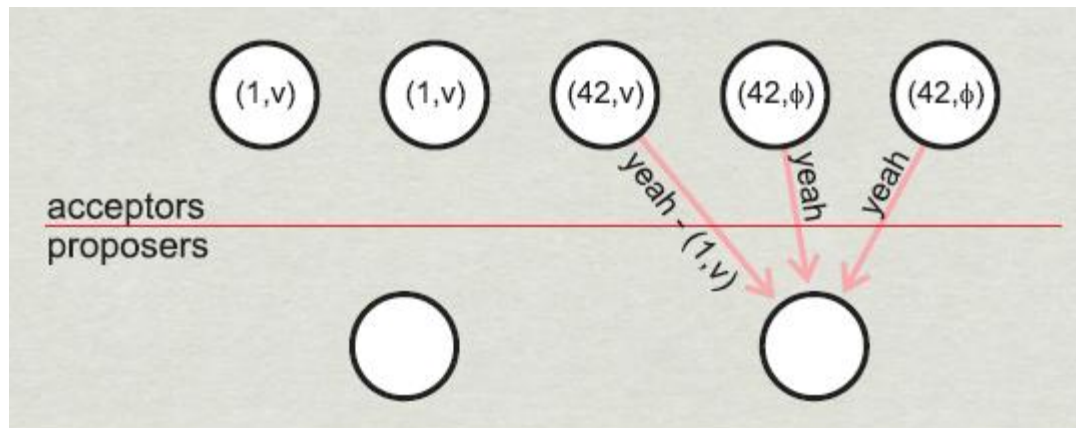
---

- ▶ In Paxos, there are **rules for proposing values**.
- ▶ If no values are returned in accept messages from phase 1;
  - ▶ proposer is free to pick any proposal value it wants.
- ▶ Otherwise;
  - ▶ proposer **has to pick a previously proposed value**;
  - ▶ the one **ACCEPTED** in the proposal with the highest number, in a previous round.









# Paxos Protocols

---

## ▶ **Phase Ia: Proposer (PREPARE)**

- ▶ A proposer initiates a PREPARE message, picking a unique, ever-incrementing value.
  - ▶ `ID = cnt++;`
  - ▶ `send PREPARE(ID)`

## ▶ **Phase Ib: Acceptor (PROMISE)**

- ▶ An acceptor receives a PREPARE(ID) message:

```
if (ID <= max_id)
    do not respond (or respond with a "fail" message)
else
    max_id = ID          // save highest ID we've seen so far
    if (proposal_accepted == true)
        respond: PROMISE(ID, accepted_ID, accepted_VALUE)
    else
        respond: PROMISE(ID)
```



---

## ▶ **Phase 2a: Proposer (PROPOSE)**

- ▶ The proposer now checks to see if it can use its proposal or if it has to use the highest-numbered one it received from among all responses:

```
did I receive PROMISE responses from a majority of acceptors?  
if yes  
    do any responses contain accepted values (from other proposals)?  
    if yes  
        val = accepted_VALUE // value from PROMISE message with the  
highest accepted ID  
    if no  
        val = VALUE          // we can use our proposed value  
    send PROPOSE(ID, val) to at least a majority of acceptors
```



---

## ► Phase 2b: Acceptor (ACCEPT)

- Each acceptor receives a PROPOSE(ID,VALUE) message from a proposer. If the ID is the highest number it has processed then accept the proposal and propagate the value to the proposer and to all the learners.

```
if (ID == max_id) // is the ID the largest I have seen so far?
    proposal_accepted = true        // note that we accepted a proposal
    accepted_ID = ID                // save the accepted proposal number
    accepted_VALUE = VALUE          // save the accepted proposal data
    respond: ACCEPTED(ID, VALUE) to the proposer and all learners
else
    do not respond (or respond with a "fail" message)
```



# Paxos Safety

---

- ▶ These simple rules bring the following property:
- ▶ *Once a value  $v$  has been decided in view  $r$ , every proposal with number  $n > r$  made by ANY PROPOSER will either:*
  - ▶ *be for value  $v$ ;*
  - ▶ *fail.*



# Why?

---

- ▶ Part of it is somewhat obvious;
- ▶ decision = majority;
- ▶ any two majorities share at least one element;
- ▶ therefore:
- ▶ after the first **decision**;
- ▶ any subsequent view, either:
- ▶ does not contain  $n/2 + 1$  acceptors;
- ▶ contains one element that accepted  $v$ .



# Why?

---

- ▶ But, that's not the end of it;
- ▶ to ensure that *v* is *proposed*, we must ensure that *v* was *always the* highest-numbered value to be accepted in any majority.
- ▶ Now, if value *v* got *accepted by a majority*;
  - ▶ then, it had the largest view number across the whole majority;
  - ▶ obvious, or it wouldn't have been accepted;
  - ▶ **less obvious: immediately after acceptance, it has the highest** number among all accepted values;



# Failure

---

- ▶ What happens after success?
- ▶ Safety: the answer is in the network;
- ▶ learners might learn it when the system becomes
- ▶ stable again.
- ▶ Acceptors or proposers may fail, responses may be lost;
- ▶ only delays progress or knowledge of consensus;
- ▶ does not affect safety



# Liveness

---

▶ **Note that:**

- ▶ multiple proposers may delay progress;
- ▶ proposer 1: completes phase 1 by proposing number  $n_1$ ;
- ▶ proposer 2: completes phase 1 by proposing  $n_2 > n_1$ ;
- ▶ proposer 1: completes phase 1 by proposing  $n_3 > n_2$ ;
- ▶ *ad infinitum*.



# Liveness

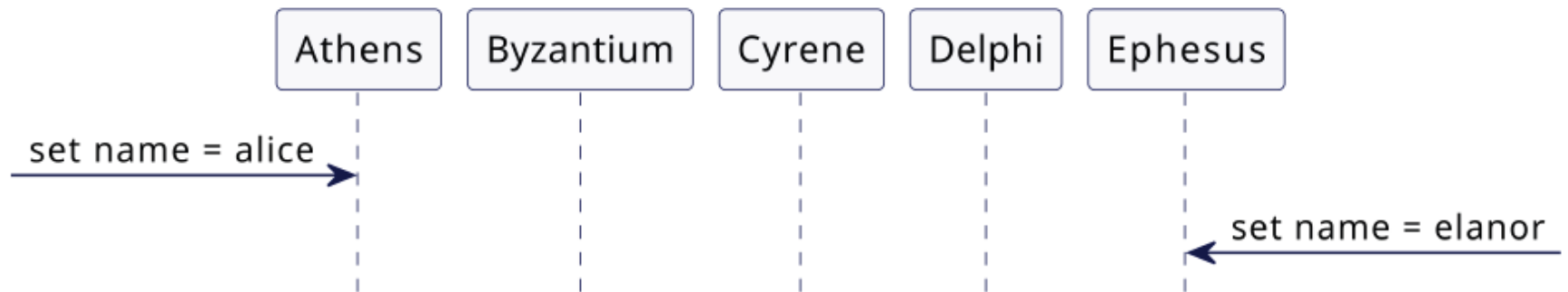
---

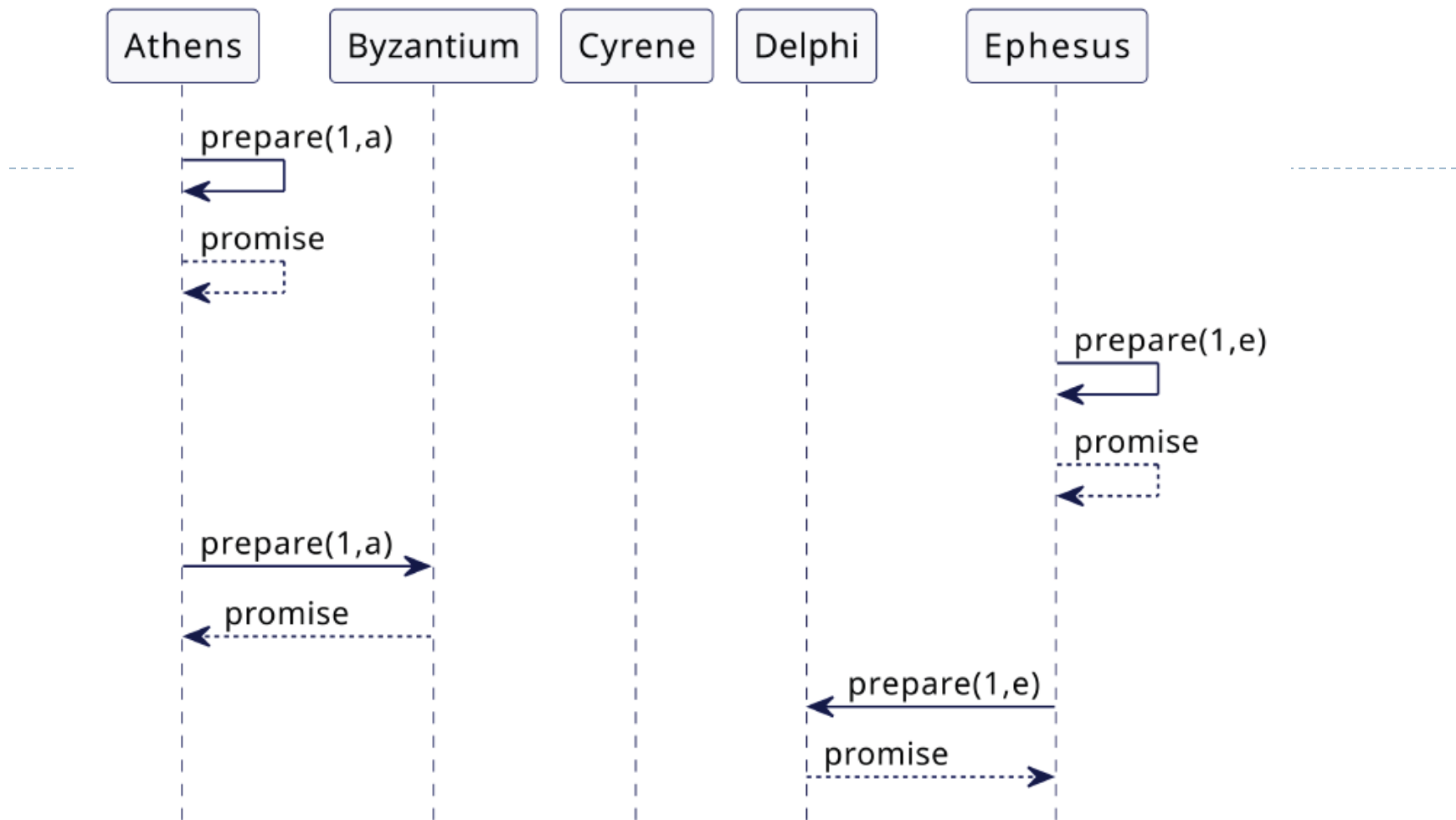
- ▶ In practice:
- ▶ people try to maintain a single proposer at any time;
- ▶ leader election, leases;
  - ▶ again, if it fails, it's okay. Algorithm does not block, though progress may be impaired;
  - ▶ if two nodes think they're leaders, it's okay as well.
- ▶ Multiple roles may be played by each node (e.g., all acceptors are learners).



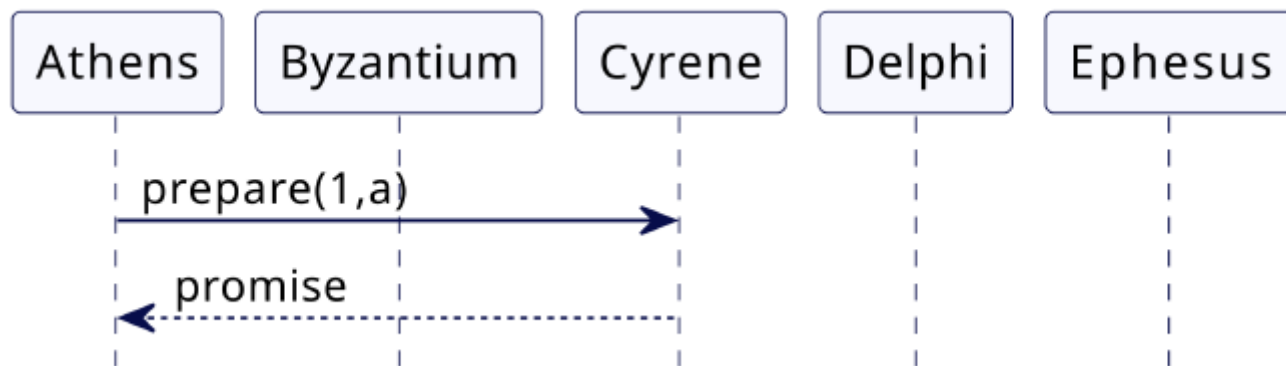
# Ilustrasi Paxos

---



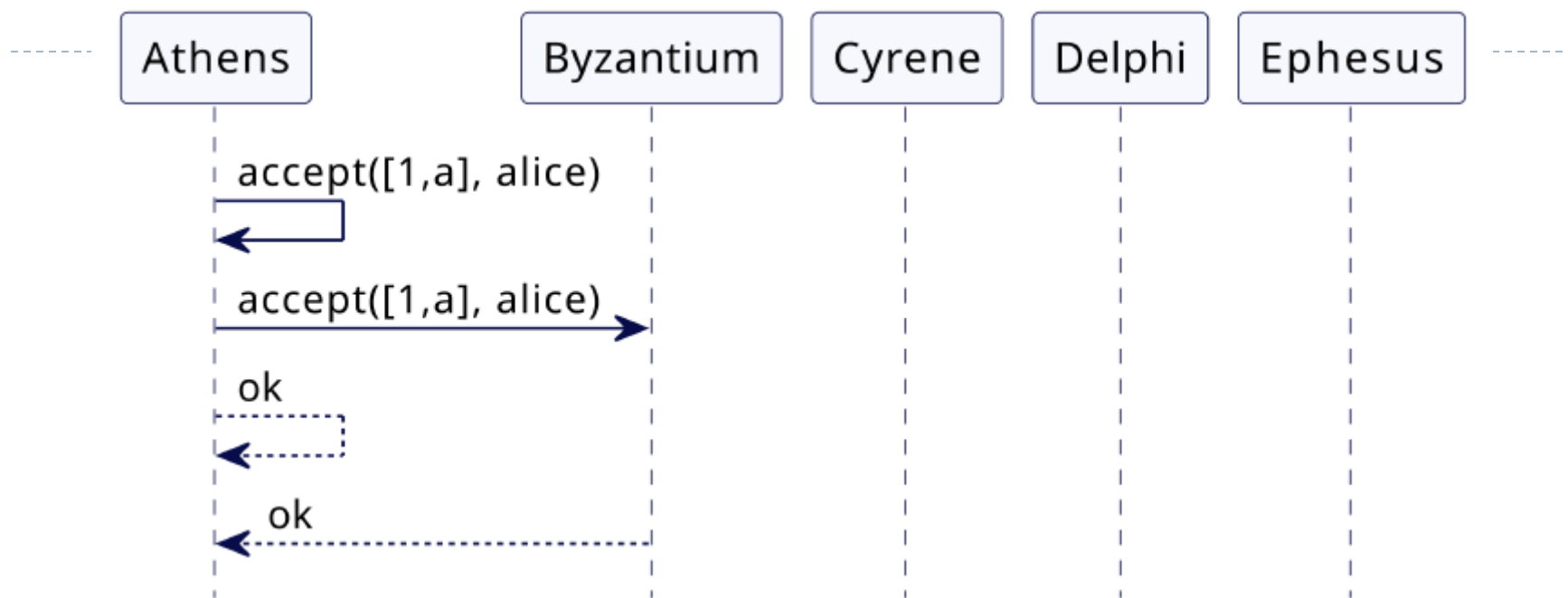


| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 1,a    | 1,a       | 0      | 1,e    | 1,e     |
| Accepted value      | none   | none      | none   | none   | none    |



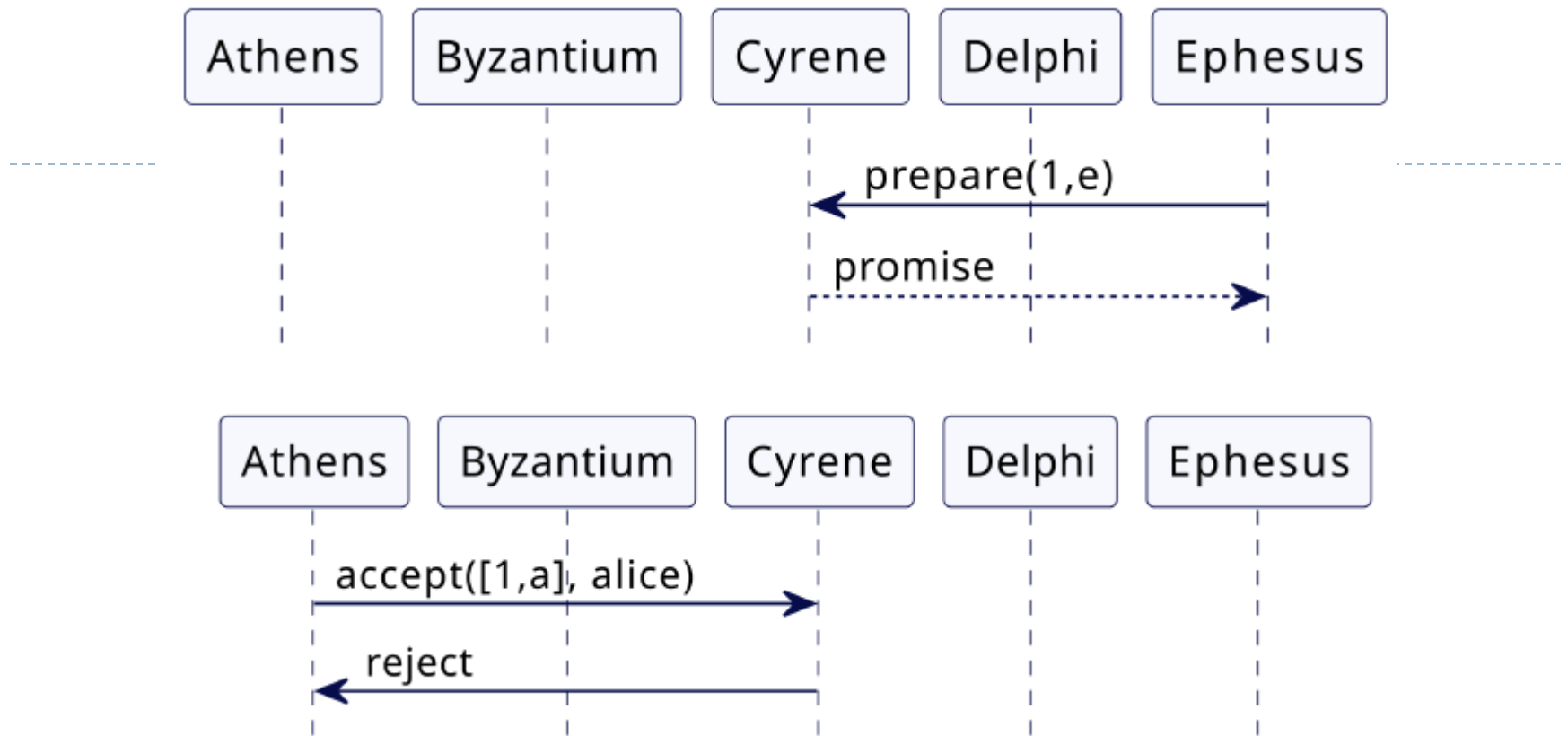
| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 1,a    | 1,a       | 1,a    | 1,e    | 1,e     |
| Accepted value      | none   | none      | none   | none   | none    |





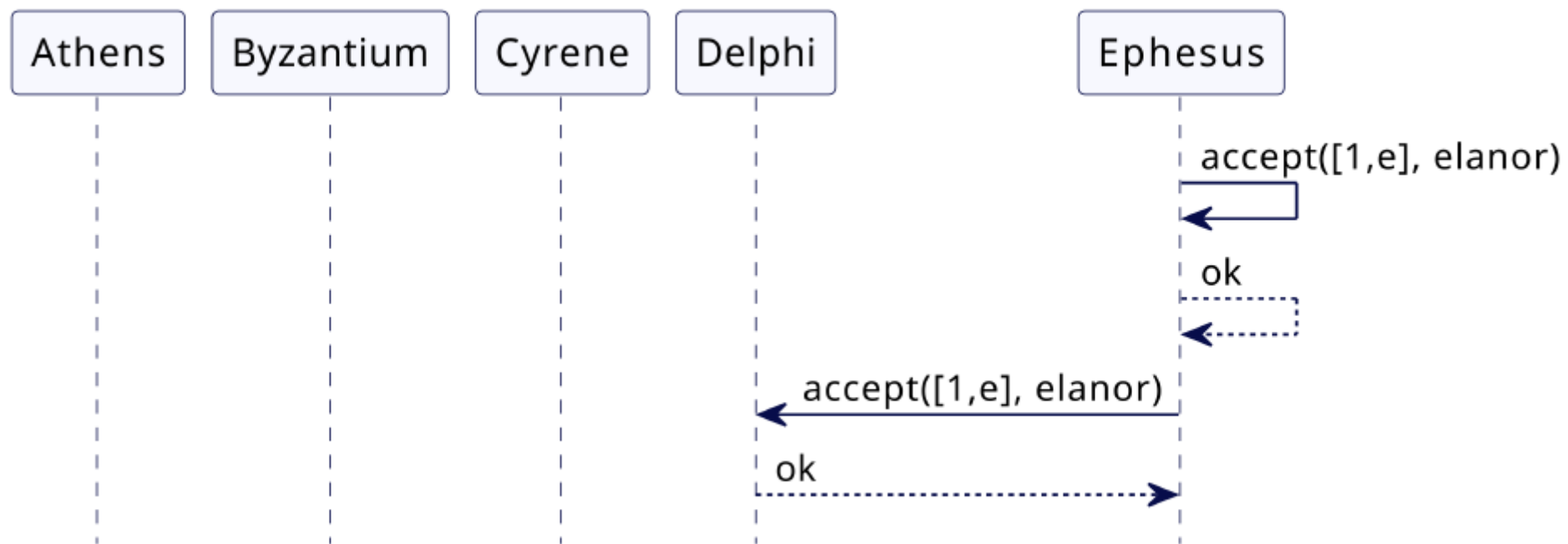
| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 1,a    | 1,a       | 1,a    | 1,e    | 1,e     |
| Accepted value      | alice  | alice     | none   | none   | none    |





| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 1,a    | 1,a       | 1,e    | 1,e    | 1,e     |
| Accepted value      | alice  | alice     | none   | none   | none    |

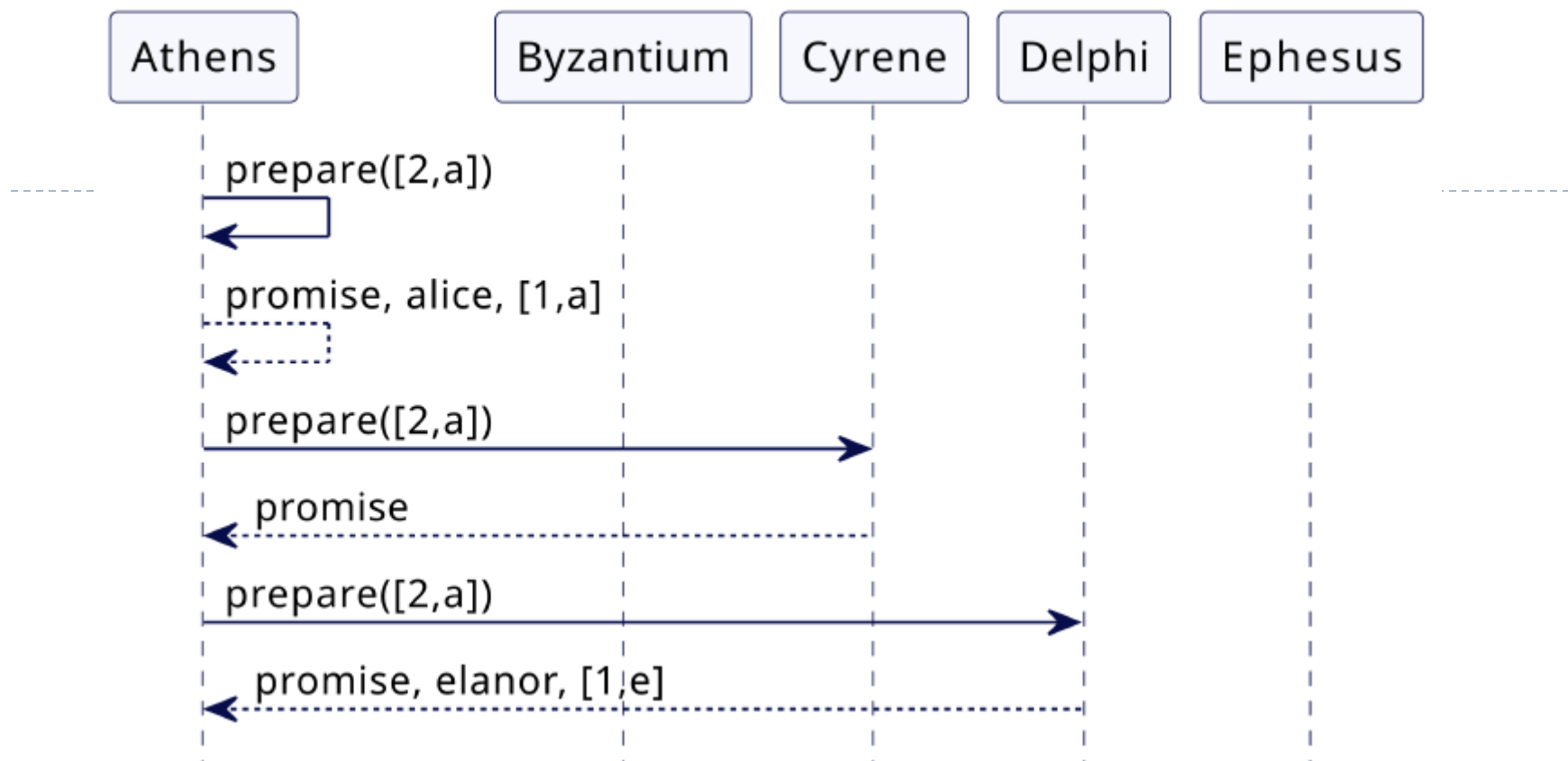




Ephesus crashes

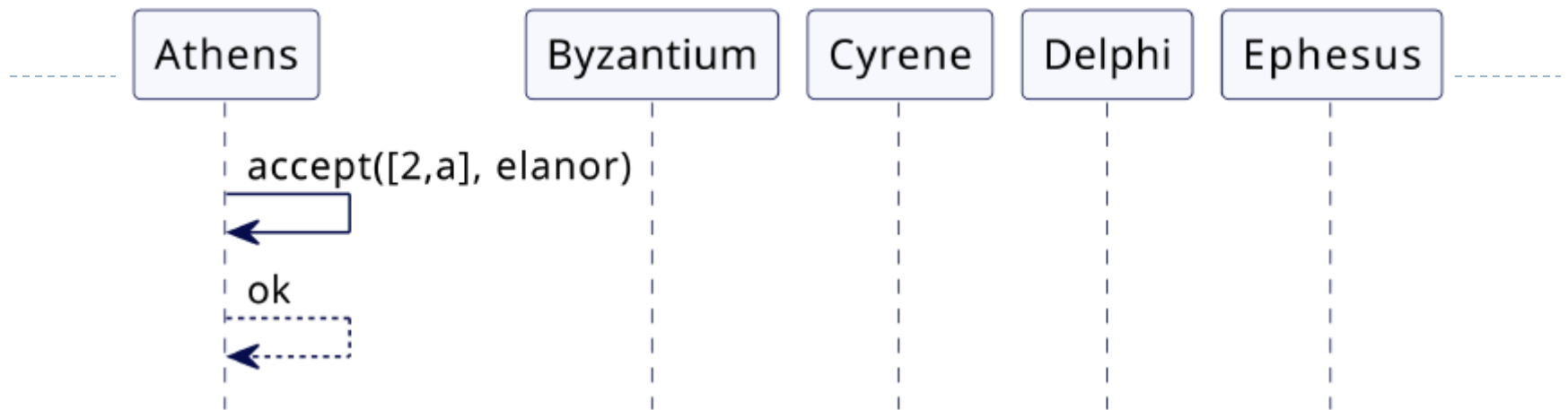
| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 1,a    | 1,a       | 1,e    | 1,e    | 1,e     |
| Accepted value      | alice  | alice     | none   | elanor | elanor  |





| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 2,a    | 1,a       | 2,a    | 2,a    | 1,e     |
| Accepted value      | alice  | alice     | none   | elanor | elanor  |



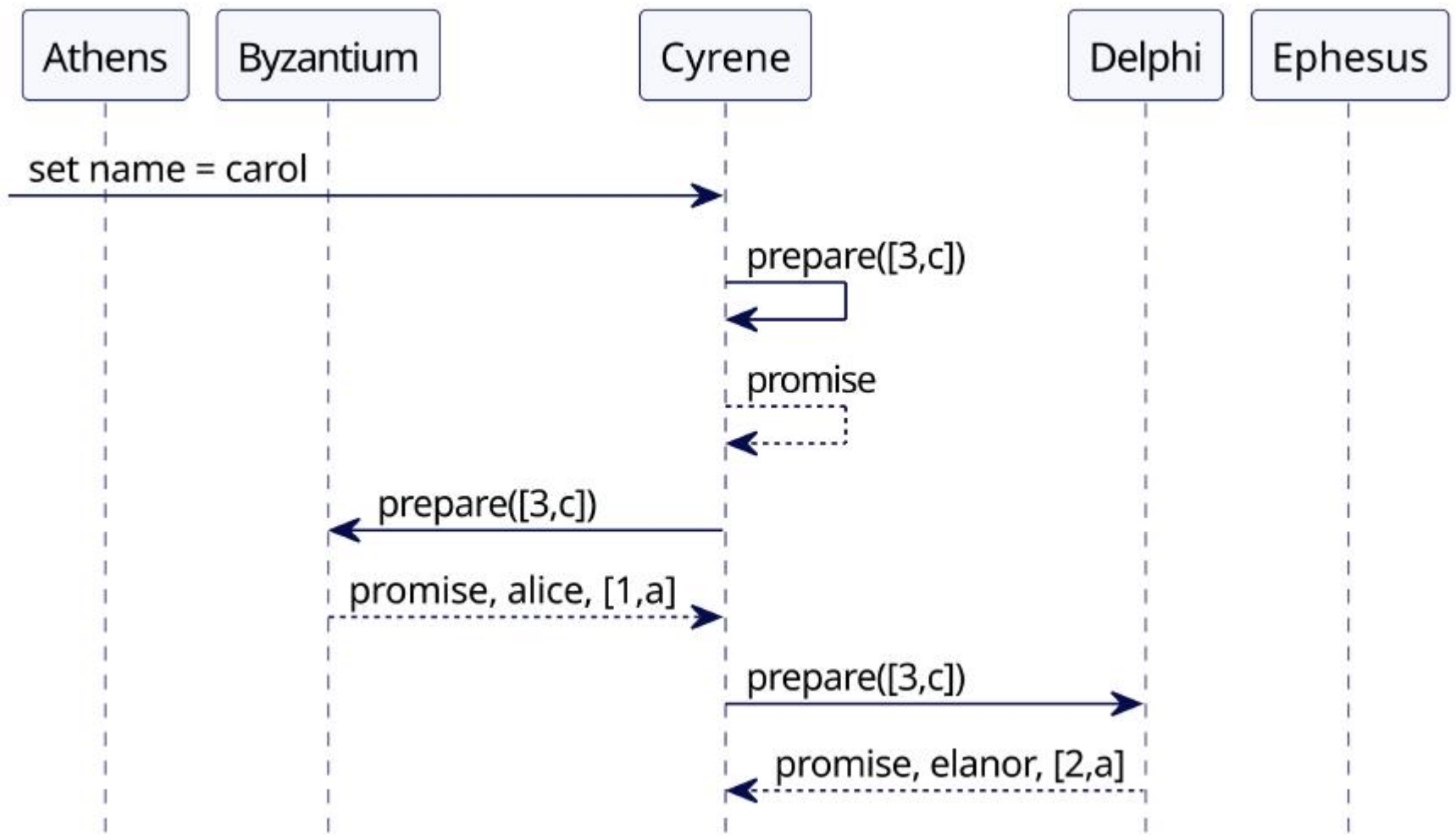


athens crash

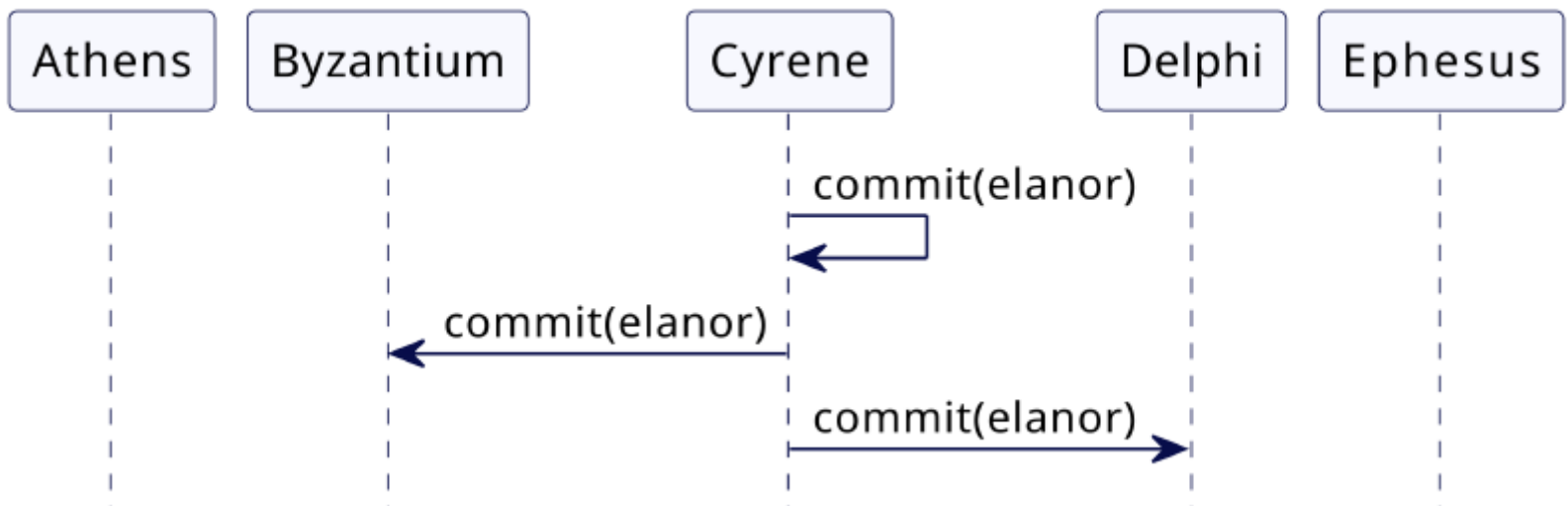
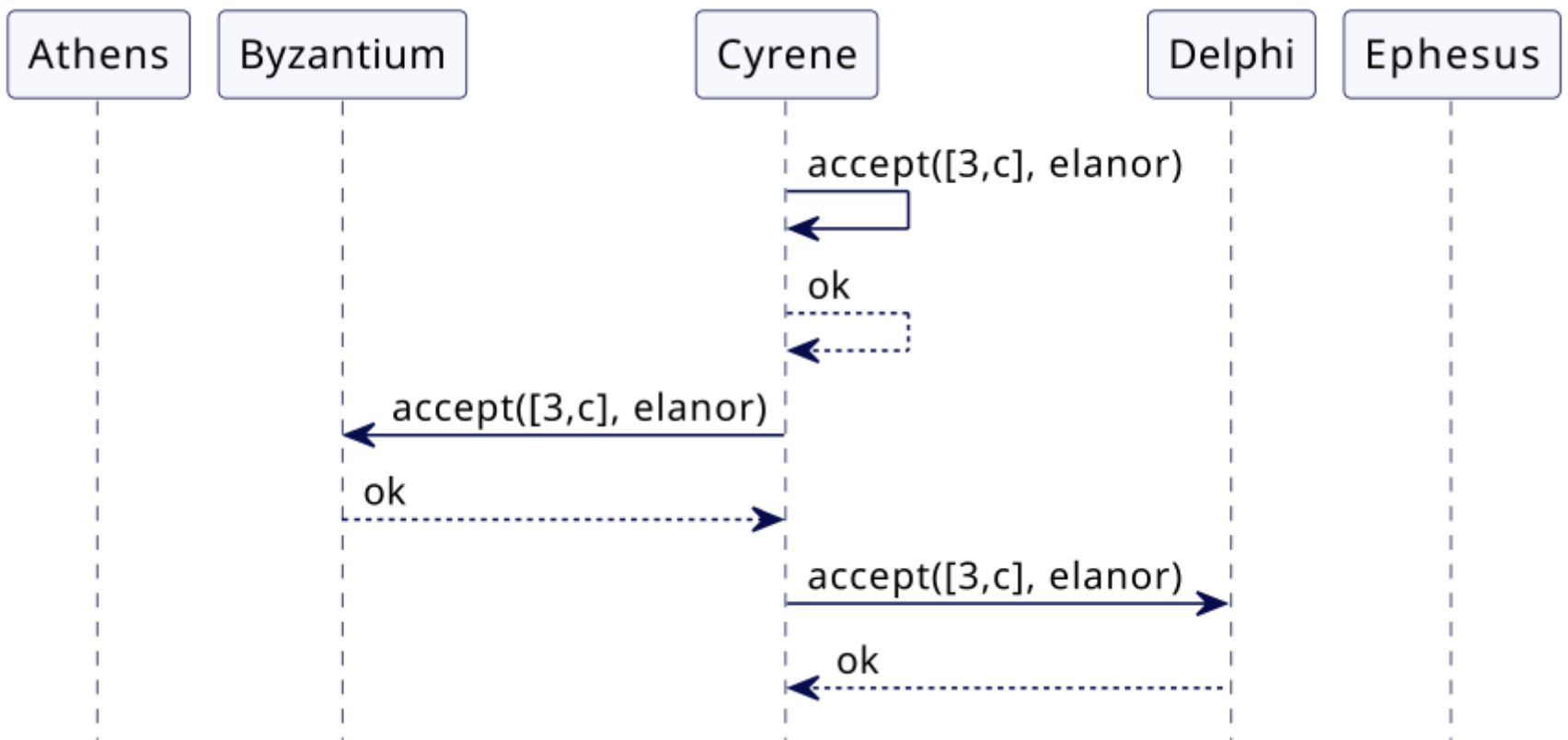
| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 2,a    | 1,a       | 2,a    | 2,a    | 1,e     |
| Accepted value      | elanor | alice     | none   | elanor | elanor  |

elanor accepted by majority, but nobody knows yet





| Node                | Athens | Byzantium | Cyrene | Delphi | Ephesus |
|---------------------|--------|-----------|--------|--------|---------|
| Promised generation | 2,a    | 3,c       | 3,c    | 3,c    | 1,e     |
| Accepted value      | elanor | alice     | none   | elanor | elanor  |



- 
- ▶ Paxos allows us to ensure consistent (total) ordering over a set of events in a group of machines
    - ▶ Events = commands, actions, state updates
  - ▶ Each machine will have the latest state or a previous version of the state
  - ▶ Paxos used in:
    - ▶ Cassandra lightweight transactions
    - ▶ Google Chubby lock manager / name server
    - ▶ Google Spanner, Megastore
    - ▶ Microsoft Autopilot cluster management service from Bing
    - ▶ VMware NSX Controller
    - ▶ Amazon Web Services
- 



# Paxos Summary

---

- ▶ To make a change to the system:
  - ▶ Tell the *proposer (leader)* the *event/command* you want to add
    - ▶ Note: these requests may occur concurrently
    - ▶ Leader = one elected proposer. Not necessary for Paxos algorithm but an optimization to ensure a single, increasing stream of proposal numbers. Cuts down on rejections and retries.
  - ▶ The proposer picks its next highest event ID and asks all the acceptors to reserve that event ID
    - ▶ If any acceptor sees it's not the latest event ID, it rejects the proposal; the proposer will have to try again with another event ID
  - ▶ When the majority of acceptors accept the proposal, accepted events are sent to learners, which can act on them (e.g., update system state)
    - ▶ Fault tolerant: need  $2k+1$  servers for  $k$  fault tolerance



# References

---

- ▶ Andrew S. Tanenbaum and Maarten Van Steen. Distributed System Principles and Paradigms. 2007
- ▶ Paul Krzyzanowski. Distributed System Lectures Notes : Consensus Paxos. Rutgers University. Fall 2015

